

Documentação do Script `jupiter sheet maker.py`

Sistema de Processamento de Planilhas Acadêmicas

13 de abril de 2026

Sumário

1	Visão Geral	2
1.1	Arquivos de Entrada	2
2	Fluxo de Processamento	2
2.1	Carregamento dos Dados	2
2.2	Normalização de Códigos de Disciplina	2
2.2.1	Normalização - Ingressantes	3
2.2.2	Tratamento de Erros	3
3	Processamento de Vagas	3
3.1	Preenchimento de Valores Nulos	3
3.2	Cálculo de Vagas por Disciplina	3
3.3	Soma de Vagas	4
4	Ajustes Especiais de Vagas	4
4.1	Adição de Ingressantes	4
4.2	Adição de Espelho	4
5	Processamento da Planilha “Outros”	5
6	Geração do Arquivo de Saída	5
6.1	Estruturação dos Dados	5
6.2	Escrita do Arquivo Excel	5
7	Tratamento de Erros na Escrita	5
7.1	Erro de Permissão	5
7.2	Erros Gerais	6
8	Códigos de Saída	6
9	Dependências	6
10	Resumo do Fluxo de Execução	6

1 Visão Geral

O script `jupyter sheet maker.py` é um programa Python desenvolvido para processar e consolidar informações de múltiplas planilhas Excel relacionadas ao sistema acadêmico Júpiter. O programa realiza a integração de dados de salas, vagas por curso, ingressantes e espelho de matrículas, gerando uma planilha consolidada com informações sobre disponibilidade de vagas por disciplina.

1.1 Arquivos de Entrada

O script recebe 5 arquivos como argumentos de linha de comando:

1. `sys.argv[1]`: Arquivo Excel com múltiplas planilhas contendo dados de salas e disciplinas por departamento (SME, SMA, SCC, SSC, Outros)
2. `sys.argv[2]`: Arquivo Excel do sistema Júpiter com informações de vagas disponíveis por disciplina
3. `sys.argv[3]`: Planilha de ingressantes
4. `sys.argv[4]`: Planilha espelho (com dados de inscritos)
5. `sys.argv[5]`: Nome do arquivo de saída

2 Fluxo de Processamento

2.1 Carregamento dos Dados

```
1 df_completo = pd.ExcelFile(sys.argv[1])
2 sheet_names = df_completo.sheet_names
3 salas = pd.read_excel(sys.argv[1], sheet_name=sheet_names[0])
4 df = pd.read_excel(sys.argv[1], sheet_name=sheet_names[1:])
```

Nesta etapa, o programa:

- Carrega o arquivo principal com todas as planilhas
- Extrai a primeira planilha como dados de salas
- Carrega as demais planilhas (SME, SMA, SCC, SSC, Outros) como um dicionário de DataFrames

2.2 Normalização de Códigos de Disciplina

O script realiza normalização dos códigos de disciplina em três arquivos diferentes (ingressantes, espelho e planilhas principais). A normalização consiste em:

1. **Remoção de espaços:** Remove espaços em branco dos códigos de disciplina
2. **Padronização do formato:** Garante que todos os códigos sigam o formato `CODIGO-TURMA`

2.2.1 Normalização - Ingressantes

```
1 for d in range(len(ingressantes['Disciplina (codigo)']):
2     if ' ' in str(ingressantes.loc[d, 'Disciplina (codigo)']):
3         ingressantes.loc[d, 'Disciplina (codigo)'] =
4             str(ingressantes.loc[d, 'Disciplina (codigo)']).
5                 replace(' ', '')
6     if '-' not in str(ingressantes.loc[d, 'Disciplina (codigo)']):
7         :
8         ingressantes.loc[d, 'Disciplina (codigo)'] =
9             f"{ingressantes.loc[d, 'Disciplina (codigo)']}-{int(
10                 ingressantes.loc[d, 'Turma'])}"
```

Esta seção processa cada linha da planilha de ingressantes, formatando os códigos de disciplina para o padrão CODIGO-TURMA.

2.2.2 Tratamento de Erros

O código implementa tratamento robusto de erros usando blocos `try-except`:

```
1 except KeyError as e:
2     coluna = str(e).strip('\')
3     msg = f"A coluna '{coluna}' nao foi encontrada no arquivo {os
4         .path.basename(sys.argv[3])}. Verifique o arquivo."
5     print(msg, file=sys.stderr)
6     sys.exit(4)
```

Quando uma coluna esperada não é encontrada, o programa exibe mensagem de erro detalhada e encerra com código de saída específico.

3 Processamento de Vagas

3.1 Preenchimento de Valores Nulos

```
1 for sheet in sheet_names_jpter[0:]:
2     vagas[sheet] = vagas[sheet].fillna(0)
```

Substitui valores vazios (NaN) por zero em todas as planilhas de vagas do Jupyter.

3.2 Cálculo de Vagas por Disciplina

Para as planilhas SME, SMA, SCC e SSC (índices 1 a 4):

1. **Normalização de colunas:** Remove quebras de linha dos nomes das colunas
2. **Formatação de disciplinas:** Padroniza códigos tanto nas planilhas de vagas quanto nas principais
3. **Identificação de colunas de vagas:** Localiza as colunas de diferentes tipos de vagas:
 - Vagas obrigatórias

- Vagas eletivas
- Vagas optativas livres
- Vagas especiais

3.3 Soma de Vagas

```

1 if df[sheet].loc[d, 'Disciplina (codigo)'] in vagas[sheet]['
  Disciplina'].tolist():
2     index = (vagas[sheet]['Disciplina'].tolist()).index(df[sheet]
  ].loc[d, 'Disciplina (codigo)'])
3     df[sheet].loc[d, 'Vagas por disciplina'] = 0
4     for i in range(len(lista2)):
5         df[sheet].loc[d, 'Vagas por disciplina'] += vagas[sheet].
          loc[index, colunas[lista2[i]]]

```

Para cada disciplina:

- Verifica se existe na planilha de vagas do Júpiter
- Localiza o índice correspondente
- Soma todas as categorias de vagas (obrigatórias, eletivas, optativas e especiais)
- Se não encontrada, define vagas como 0

4 Ajustes Especiais de Vagas

O script verifica a coluna Observações para aplicar ajustes adicionais:

4.1 Adição de Ingressantes

```

1 if "Ingressantes" in str(df[sheet].loc[d, 'Observacoes']):
2     if df[sheet].loc[d, 'Disciplina (codigo)'] in ingressantes['
  Disciplina (codigo)'].tolist():
3         index = (ingressantes['Disciplina (codigo)'].tolist()).
          index(df[sheet].loc[d, 'Disciplina (codigo)'])
4         df[sheet].loc[d, 'Vagas por disciplina'] += ingressantes.
          loc[index, 'Ingressantes']

```

Se a observação contém a palavra “Ingressantes”, adiciona o número de ingressantes ao total de vagas.

4.2 Adição de Espelho

```

1 if "Espelho" in str(df[sheet].loc[d, 'Observacoes']):
2     if df[sheet].loc[d, 'Disciplina (codigo)'] in espelho['
  Disciplina (codigo)'].tolist():
3         index = (espelho['Disciplina (codigo)'].tolist()).index(
          df[sheet].loc[d, 'Disciplina (codigo)'])

```

```

4         df[sheet].loc[d, 'Vagas por disciplina'] += espelho.loc[
            index, 'Inscritos']

```

Se a observação contém a palavra “Espelho”, adiciona o número de inscritos do espelho ao total de vagas.

5 Processamento da Planilha “Outros”

A planilha “Outros” (índice 5) recebe tratamento especial:

1. Normalização dos códigos de disciplina (similar às outras planilhas)
2. Processamento de vagas considerando as planilhas do Júpiter de índice 4 em diante
3. Aplicação dos mesmos ajustes de ingressantes e espelho
4. Tratamento especial para valores NaN: mantém como 0 se não encontrada

6 Geração do Arquivo de Saída

6.1 Estruturação dos Dados

```

1 dfs = {
2     "Salas": salas,
3     "SME": df['SME'],
4     "SMA": df['SMA'],
5     "SCC": df['SCC'],
6     "SSC": df['SSC'],
7     "Outros": df['Outros']
8 }

```

Organiza os DataFrames processados em um dicionário com nomes das planilhas.

6.2 Escrita do Arquivo Excel

```

1 file_path = os.path.join(os.getcwd(), "Saídas da Interface", "
    Planilhas de Dados", full_name)
2
3 with pd.ExcelWriter(file_path, engine='openpyxl') as writer:
4     for sheet_name, df in dfs.items():
5         df.to_excel(writer, sheet_name=sheet_name, index=False)

```

Gera arquivo Excel no diretório Saídas da Interface/Planilhas de Dados com todas as planilhas processadas.

7 Tratamento de Erros na Escrita

7.1 Erro de Permissão

```

1 except PermissionError as e:
2     if e.errno == 13:
3         traceback.print_exc(file=sys.stderr)
4         sys.exit(2)

```

Captura erros quando o arquivo está aberto ou bloqueado (código de saída 2).

7.2 Erros Gerais

```

1 except Exception as e:
2     traceback.print_exc(file=sys.stderr)
3     sys.exit(1)

```

Captura qualquer outro erro, imprime o traceback completo e encerra com código 1.

8 Códigos de Saída

O programa utiliza diferentes códigos de saída para indicar tipos específicos de erro:

Código	Significado
0	Execução bem-sucedida
1	Erro geral não especificado
2	Erro de permissão (arquivo aberto/bloqueado)
3	Outro erro de permissão
4	Coluna não encontrada em algum arquivo de entrada

Tabela 1: Códigos de saída do programa

9 Dependências

O script requer as seguintes bibliotecas Python:

- **pandas**: Manipulação de planilhas Excel e DataFrames
- **openpyxl**: Engine para leitura/escrita de arquivos .xlsx
- **os**: Operações de sistema de arquivos
- **sys**: Acesso a argumentos da linha de comando
- **traceback**: Formatação de mensagens de erro

10 Resumo do Fluxo de Execução

1. Carregamento de todos os arquivos de entrada
2. Normalização dos códigos de disciplina em todos os arquivos

3. Preenchimento de valores nulos nas planilhas de vagas
4. Para cada departamento (SME, SMA, SCC, SSC):
 - Cálculo da soma de vagas (obrigatórias + eletivas + optativas + especiais)
 - Adição de vagas de ingressantes (se aplicável)
 - Adição de vagas do espelho (se aplicável)
5. Processamento especial da planilha “Outros”
6. Geração do arquivo Excel consolidado
7. Tratamento de erros e retorno de código de saída apropriado